CRYPTOGRAPHY | FINAL PROJECT REPORT

Bitcoin Real-Time Cryptographic Dashboard

A live monitor of Proof-of-Work, block headers, network difficulty and an AI forecast of the next difficulty adjustment.

AUTHOR	María Muñoz Nadales
COURSE	Cryptography (Asignatura de Criptografía)
DELIVERABLE	Final Report – Project Documentation
DATE	May 2026

1 Project Overview

The *Bitcoin Real-Time Cryptographic Dashboard* is an interactive Streamlit application that visualises live data from the Bitcoin network and explains, step by step, the cryptographic primitives that make the chain trustworthy. It connects to public Bitcoin APIs (`blockchain.info` and `Blockstream Esplora`) and is organised in four core modules (**M1**–**M4**) plus four advanced ones (**M5**–**M8**). The dashboard is centred on three questions that map directly to the topics of the course: **(i)** how is each block protected by hashing, **(ii)** how does the network adapt difficulty so that a new block is produced roughly every ten minutes, and **(iii)** how can a simple AI model anticipate the next difficulty adjustment.

2 Cryptographic Metrics Displayed

The dashboard displays a set of metrics that together describe the security state of the Bitcoin chain at a given moment. Each one is computed in real time and explained inside the UI.

2.1 Block hash and double-SHA-256

For every new block, module M2 fetches the raw 80-byte block header and recomputes the hash locally as $H = \mathrm{SHA256}(\mathrm{SHA256}(\mathrm{header}))$, reversing the output to display order. This is the same operation Bitcoin miners repeat billions of times per second. The dashboard parses the six header fields (`version`, `previous_block_hash`, `merkle_root`, `timestamp`, `bits`, `nonce`) and verifies that the recomputed hash matches the value reported by the API, proving header integrity client-side.

2.2 Target, leading zero bits and Proof of Work

The compact `bits` field is unpacked into the full 256-bit target T through $T = c \cdot 2^{8(e-3)}$, where e is the exponent and c the coefficient. A block is valid if and only if $H < T$. The UI shows the number of *leading zero bits* of H, which is the most intuitive proxy for the work spent: each extra leading zero requires, on average, twice as many hash attempts. This is the practical realisation of Adam Back's *hashcash*-style proof of work referenced in the Bitcoin whitepaper [**?**].

2.3 Difficulty and estimated hash rate

Difficulty D is reported per block. Combined with the average inter-block time $\bar{t}$ measured over the last ten blocks, module M1 estimates the network hash rate as

$$R \;=\; \frac{D \cdot 2^{32}}{\bar{t}}$$

This formula comes from the fact that, on average, $D \cdot 2^{32}$ hashes must be tried to find a valid block [**?**]. The dashboard also classifies network stress (LOW / MEDIUM / HIGH) by comparing $\bar{t}$ to the 600-second target.

2.4 Difficulty history and re-targeting

Module M3 plots the last year of difficulty values fetched from `blockchain.info/charts/difficulty`. Bitcoin re-targets every 2016 blocks (∼two weeks); the chart reveals these step-like adjustments and the long-term trend of network commitment. KPIs include latest, previous, minimum and maximum difficulty over the window.

2.5 Merkle proof and 51% attack economics (advanced)

Module M5 reconstructs the Merkle tree of the latest block from its transaction IDs and walks the path from any chosen transaction up to the `merkle_root`, validating it with double-SHA-256. Module M6 implements Nakamoto's catch-up probability for an attacker controlling a fraction q of the hash rate after z confirmations [**?**, §11]:

$$P(z,q) \;=\; 1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!} \left(1 - \left(\tfrac{q}{p}\right)^{z-k}\right), \quad \lambda = z \cdot \tfrac{q}{p}.$$

This curve, together with an electricity-cost model, lets the user explore how many confirmations are economically safe.

3 AI Component

3.1 Model and rationale

The chosen AI approach is a **Predictor** for the next Bitcoin difficulty value, implemented in module M4. The primary model is an **ordinary least-squares linear regression** fitted on the last 180 points of the difficulty time series, returning slope β and intercept α so that $\hat{D}_{n+1} = \alpha + \beta\, n$. A second model (M7) uses **simple exponential smoothing** with weight α, computing $S_t = \alpha\, D_t + (1-\alpha)\, S_{t-1}$ and using the last S as the next-step forecast.

Linear regression was selected as the main predictor for three reasons:

- Bitcoin difficulty is a piecewise-monotonic series anchored to a constant ten-minute target. Across a multi-week horizon the dominant component is a linear drift that tracks miner deployments, which a first-order regression captures with very few parameters.
- It is fully transparent: slope and intercept can be inspected by students of the course, which matches the pedagogical goal of the dashboard.
- It runs in milliseconds and has no external dependency, so it can be re-trained on every refresh without slowing the UI.

3.2 Evaluation results

Both models are evaluated with a rolling **one-step-ahead backtest**: at each index $t \geq 8$, the model is retrained on $D_{0..t-1}$ and the absolute error $|\hat{D}_t - D_t|$ is recorded. The reported metric is the *Mean Absolute Error* (MAE) over the rolling window.

Model	**Rolling MAE**	**Last-step error**
Linear regression (180 d)	$\sim 1.4 \times 10^{12}$	0.18 % relative
Exponential smoothing (α=0.35)	$\sim 2.1 \times 10^{12}$	0.42 % relative

The linear model produces a smaller MAE because the difficulty series is strongly trending; exponential smoothing reacts more slowly to the regular 2016-block step adjustments. The dashboard renders a *confidence band* proportional to the predicted percentage change, and labels the forecast as High / Medium / Low confidence based on the magnitude of $\hat{D}_{n+1} - D_n$. On the most recent epoch the model produced a +0.31 % expected change with a backtest within 0.2 % of the realised value, classified as **High confidence**.

4 Conclusions

The dashboard turns abstract concepts (hash, target, Merkle tree, attack probability) into measurable, live quantities. The cryptographic layer is verified locally rather than trusted from the API: every header is re-hashed, every Merkle root is rebuilt from raw transaction IDs, and every target is decoded from `bits`. The AI layer is intentionally simple to remain interpretable, but the side-by-side comparison in M7 shows that even classical statistical models can track Bitcoin difficulty within sub-percent error over a one-step horizon. Future work would extend M4 with an ARIMA/Prophet model that explicitly captures the 2016-block re-targeting period.

References

[1] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
https://bitcoin.org/bitcoin.pdf

[2] Bitcoin Wiki, *Difficulty*.
https://en.bitcoin.it/wiki/Difficulty

[3] Blockstream, *Esplora HTTP API documentation*.
https://github.com/Blockstream/esplora/blob/master/API.md

[4] National Institute of Standards and Technology, *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.
https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf